

HW: ECC

CYB-402

Tyler Vander Mooren

1.

- Why ECC was developed
 - ECC was created because public key systems like RSA are requiring increasingly larger key sizes to maintain security which slows computation. ECC methods provide equivalent security with far smaller key sizes which reduces the computation needed.
- Security Advantage
 - The major advantage of ECC is the key size as a 256-bit elliptic curve key provides roughly the same security as a 3072-bit RSA key. Using a smaller key size reduces power consumption and computation time making ECC more ideal for performance and resource limited systems.
- Non-Singular Curves
 - For an elliptic curve to be secure it must be non-singular meaning it cannot have cusps, sharp points or self-intersections. Singular curves ruin the math structure needed for cryptographic operations. Making sure the curve is smooth guarantees that addition and multiplication behave correctly and securely.
- Elliptic Curves over Finite Fields
 - ECC takes place in finite fields such as $\mathbb{Z}_p$ which means all calculations are done modulo a prime p . This prevents rounding errors and makes sure every point on the curve is defined.
- Reliable Computation
 - Because ECC uses modular arithmetic it makes every operation produce predictable and accurate results. This prevents numerical errors that would make encryption or signatures fail. This reliability makes ECC suitable for cryptographic systems.
- Group law on Elliptic Curves
 - The points on an elliptic curve form an abelian group under a special “addition” operation. To add two points, you draw a line through them and find the third intersection on the curve and then reflect it across the x-axis to get the result.
- Repeated Addition and Scalar Multiplication
 - Scalar multiplication means adding a point P to itself k times ($k \cdot P$). This operation can be computed quickly using doubling methods. The entire security of ECC comes from the fact that reversing this operation to recover K is extremely difficult.
- Difficulty of Solving

- The challenge is to find k from P and $k*P$ but even with powerful computers there are no known shortcuts or efficient algorithms to solve the Elliptic Curve Logarithm Problem (ECDLP) on a properly chosen curve. So, this difficulty is what gives ECC its strong security foundation.
- ECC Application with Diffie-Hellman
 - ECC supports secure key exchange through Elliptic Curve Diffie-Hellman. It uses the difficulty of solving k from $k*P$ to generate the secure shared secrets and is used in TLS, messaging systems and authentication systems. With ECC involved it makes a faster and more efficient version of the Diffie-Hellman key exchange.
- ECC in Digital Signatures
 - ECC also supports Elliptic Curve Digital Signature Algorithm (ECDSA) which is used to verify authenticity and integrity of messages or software. Since again it's a smaller key the ECDSA signatures are much smaller than RSA signatures while providing the same security. Making it preferred in modern systems since it does faster verification and less overhead in certificates.

2. Although its name is Elliptic Curve Cryptography it really doesn't have anything to do with an ellipse in the usual geometric sense. Ellipses are formed by quadratic curves while the elliptic curves are always cubic. The actual naming comes from their relationship to elliptic integrals in mathematics. Where the elliptic integral is used to get the arc length of an ellipse. And in this case an elliptic curve is a cubic equation with some fixed values for the parameters a and b . Usually written like this $y^2 = x^3 + ax + b$. These fixed values are chosen so the curve is smooth and doesn't have any cusps. This is enforced by a condition on a and b where the discriminant must not be zero. When the discriminant is not zero it guarantees the curve will have no cusps, sharp points or self-intersections.

3. The group law on an elliptic curve is built around the geometric idea of adding two points on the curve by drawing a line through them. For example, you take any two points A and B on an elliptic curve and draw a straight line through them. That line will usually intersect at a third point which is C . To define the sum of A and B we take that third point C and then reflect it across the x -axis. Now this reflected point becomes $A + B$. Now if we want to add a point to itself $2A$, the idea is similar but instead of two different points we draw a tangent line at point A . The tangent will then intersect the curve at a second point where you reflect that intersection across the x -axis to get $2A$. We can also find the inverse of a point by reflecting it across the x -axis, which is just the negative of the original point. The identity element called the point at infinity behaves like zero in the group and is the point where all lines intersect the curve. This process is what turns the set of points on the curve into an Abelian group.

4. The fundamental reason ECC can use much shorter keys than RSA is that simply the math problem behind ECC is harder than the math problem behind RSA even when the key sizes are small. Since RSA's security depends on the difficulty of factoring large prime integers this key needs to become larger as computers become faster and factoring algorithms improve. RSA gives

attackers more mathematical patterns they can potentially exploit. While ECC relies on figuring out k from P and $k*P$ which grows in difficulty exponentially faster than factoring does. Also, there are no known shortcuts or efficient algorithms for solving the Elliptic Curve Discrete Logarithm Problem on well chosen curves. Giving attackers far less usable patterns to use to crack ECC.

5. ECDSA depends heavily on the value of k being random because the signature equation directly includes it in a way that mathematically links k to the private key.

$$\text{sig}_1 = (K \times G)_x \bmod n$$

$$\text{sig}_2 = K^{-1} * (H + X * \text{sig}_1) \bmod n$$

So, if an attacker can guess k even partially, they can work backwards to solve for the private key because all the other parts are public. In the case where the k is ever reused in two different signatures an attacker would be able to compute the k immediately and be able to recover the private key. This is why ECDSA requires k to be unpredictable and never reused, basically a weak or repeated k hands the attacker the private key on a silver platter.

6.b.

```
mult_inverse(value,modulus)
    a = value
    b = modulus
    inv_prev = 1
    inv_curr = 0
    While b != 0
        q = a div b

        temp = a
        a = b
        b = temp mod b

        temp = inv_curr
        inv_curr = inv_prev - q * inv_curr
        inv_prev = temp
    end

    if a != 0
        return "no multiplicative inverse"
    else
        return inv_prev + modulus mod modulus
```

```

odd(curve, point, Q, modulus)
    if point = "infinity" and Q = "infinity"
        return "infinity"
    else if point = "infinity"
        return "infinity"
    else if Q = "infinity"
        return "infinity"
    if  $x_1 = x_2$  and  $y_1 = y_2$ 
        numer =  $3 * x_1^2 + a$ 
        denom =  $2 * y_1$ 
    else if  $x_1 = x_2$ 
        return "infinity"
    else
        numer =  $y_2 - y_1$ 
        denom =  $x_2 - x_1$ 

    denom_inv = mult_inverse(denom, modulus)
    alpha = (numer * denom_inv) mod modulus

     $x_3 = (\alpha^2 - x_1 - x_2) \bmod \text{modulus}$ 
     $y_3 = (\alpha * (x_1 - x_3) - y_1) \bmod \text{modulus}$ 
    return (x3,y3)

```

```

k_times_point(curve, point, k, modulus)
    if  $k \leq 0$ 
        exit with error
    if point = "infinity"
        return "infinity"
    if  $k = 1$ 
        return point
    if  $k = 2$ 
        return add(curve, point, point, modulus)
    if k is odd
        return add(curve, point, k_times_point(curve, point, k-1, modulus), modulus)
    if k is even
        else
            doublePoint = add(curve, point, point, modulus)
            return k_times_point(curve, doublePoint, k/2, modulus)

```

```

on_curve(curve, point, mod)

```

```
x = point.x
y = point.y
a = curve.a
b = curve.b
```

```
left = (y^2) mod modulus
right = (x^3 + a * x + b) mod modulus
if left = right
    return True
else
    return False
```

```
get_point_on_curve(curve, modulus)
    choose random x in [1, modulus - 1]
    y = 0
    trial = 0
```

```
While True
```

```
    trial = trial + 1
    if trial  $\geq$  2 * modulus
        break
```

```
    right_side(x^3 + a * x + b) mod modulus
    if right_side = 1
        y = 1
        break
```

```
    factors = factorize(right_size)
    if (length of factors = 2) and (factors[0] = factors[1])
        y = factors[0]
        break
```

```
    choose new random x in [1, modulus - 1]
```

```
end
```

```
if y = 0
    exit with error "no point found"
else
    return (x,y)
```

```

choose_curve_params(modulus, bits)
    While True
        a = random_integer_with_bit_length(bits)
        b = random_integer_with_bit_length(bits)

        if  $(4 * a^3 + 27 * b^2) \bmod \text{modulus} = 0$ 
            continue
        break
    end
    return(a,b)

```

```

compare(point1, point2)
    if point1.x = point2.x
        if point1.y > point2.y
            return 1
        else if point1.y < point2.y
            return -1
        else
            return 0
    else if point1.x > point2.x
        return 1
    else
        return -1

```

```

display(all_points)
    infinity_list = []
    numeric_points = []

    for each point in all_points
        if point = "infinity"
            add point to infinity_list
        else
            x = point.x
            y = point.y
            add (x,y) to numeric_points
    end

    sort numeric_points using compare()
    result_list = numeric_points + infinity_list

```

```
print result_list
```